Lecture 3: Training and Fine-Tuning LLMs

Large Language Models in Finance

Juan F. Imbet

Paris Dauphine – PSL University

Outline

Financial Data: A Tiny Slice of the Web

THE BIGGER PICTURE

Financial text constitutes roughly **0.5%** of a typical filtered web crawl. A model that has seen the distribution of the web will be fluent in casual prose and weak on financial contracts.

Four key financial text sources:

1. **EDGAR** — 10-K, 10-Q, 8-K, proxy filings; full regulatory English; public from 1996 to present
2. **Earnings-call transcripts** — spoken-register financial language, Q&A with management; available via commercial vendors or scraping

Financial Data Sources (continued)

3. **Bloomberg news** — licensed; high-density, entity-linked financial news with multi-decade coverage
4. **Alternative data** — satellite-imagery metadata, credit-card transaction summaries, job-posting feeds, shipping manifests

Consequence

Low financial representation $\Rightarrow$ high tokeniser fertility on financial text $\Rightarrow$ longer sequences $\Rightarrow$ higher cost and lower accuracy.

Tokenisation: BPE and Fertility

Byte-pair encoding (BPE) iteratively merges the most frequent adjacent symbol pair until the target vocabulary size is reached.

Fertility formula:

$$F(\tau, C) = \frac{\text{tokens produced by } \tau \text{ on corpus } C}{\text{whitespace-separated words in } C}$$

Fertility ≈ 1.0 means efficient coverage; fertility > 1.5 means heavy fragmentation of domain vocabulary.

Text type	**Fertility (GPT-2 BPE)**	**Effect**
Standard English prose	1.10–1.20	Efficient
Financial regulatory text	1.40–1.60	Fragmented

Words like `CUSIP`, `EBITDA`, and `ISDA` may be split into 3–5 sub-tokens by a general BPE tokeniser, each consuming an attention slot. Training a domain-specific tokeniser alongside domain-specific weights reduces fertility and shortens effective sequence lengths.

Data Quality Pipeline

Raw web crawl data is $\approx 90\%$ noise. A standard pipeline applies four stages:

1. **Language identification** — fastText classifier assigns each document a language label; non-target languages discarded
2. **Heuristic filters** — punctuation rate, symbol-to-word ratio, minimum token count, repeated n-gram detection
3. **Perplexity filtering** — small KenLM reference model scores each document; high-perplexity outliers removed
4. **MinHash deduplication + PII scrubbing** — near-duplicate removal by Jaccard similarity; removal of emails, phone numbers, SSNs

Data Quality: MinHash Deduplication

MinHash: Jaccard similarity definition

$$J(A, B) = \frac{|S(A) \cap S(B)|}{|S(A) \cup S(B)|}$$

where $S(d)$ is the set of k-gram shingles of document d. Documents with $J > 0.8$ are near-duplicates; all but one copy is removed.

Why deduplication matters most

Duplicated training data causes memorisation, not generalisation.
Financial press releases are reproduced verbatim by hundreds of portals.

Pre-training Objectives: CLM, MLM, Span Corruption

Objective	**Loss**	**Best suited for**
CLM (causal LM)	$-\sum_t \log P(x_t \mid x_{<t})$	Generation, chat, decoder-only (GPT)
MLM (masked LM)	$-\sum_{t\in\mathcal{M}} \log P(x_t \mid x_{\setminus\mathcal{M}})$	Classification, NER, encoder-only (BERT, FinBERT)
Span corruption (T5)	Reconstruct masked spans from sentinels	Seq2seq, QA, summarisation

CLM uses causal (lower-triangular) attention masking. MLM attends *bidirectionally* to all unmasked tokens. Span corruption replaces runs of consecutive tokens with a single sentinel `<extra_id_k>` and trains the decoder to produce all spans in order.

Finance rule of thumb

Classification / extraction tasks $\rightarrow$ MLM encoder (FinBERT).

Chinchilla Scaling Laws: Setup

Hoffmann et al. (2022) fit the empirical loss of an autoregressive LM as:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

- N = number of trainable parameters
- D = number of training tokens
- E = irreducible entropy floor
- $\alpha, \beta > 0$ govern the decay rate of each term

Compute constraint:

$$C \approx 6ND$$

(factor 6: 2 for forward pass, 4 for backward pass and parameter update, standard mixed-precision accounting)

The question

Given a fixed FLOP budget C, what is the optimal split (N^*, D^*)?

Chinchilla Scaling Laws: Result

Minimising $L(N, D)$ subject to $C = 6ND$ gives:

$$N^*(C) \propto C^{\beta/(\alpha+\beta)}, \qquad D^*(C) \propto C^{\alpha/(\alpha+\beta)}$$

With Chinchilla estimates $\alpha \approx 0.34$, $\beta \approx 0.28$:

$$\boxed{\frac{D^*}{N^*} \approx 20}$$

Worked example: 7B-parameter model

	GPT-3 (175B)	Chinchilla-optimal 7B
Parameters	175B	7B
Training tokens	300B	140B
D/N ratio	1.7	20
Status	Over-parameterised	Optimal

Practical implication

The Adaptation Spectrum

Method	**Params updated**	**Data needed**	**Best use case**
Zero-shot	0	0	Well-defined tasks in training distribution
Few-shot (ICL)	0	1–100 examples	Novel formats; no gradient compute
PEFT (LoRA)	0.1–1%	1K–100K	Domain shift; resource-constrained
Full fine-tuning	100%	100K+	Max in-domain accuracy; large data budget

The Adaptation Spectrum: The Adaptation Tax

THE BIGGER PICTURE

Adaptation tax. Full fine-tuning on a narrow domain can degrade performance on adjacent tasks — a real risk when a deployed finance LLM must also handle general client queries. The art of adaptation is buying in-domain accuracy without surrendering the breadth that made the base model useful.

LoRA: Low-Rank Adaptation

UNDER THE HOOD

For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA sets:

$$W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

W_0 is frozen. Only B and A are trained. Trainable parameters per layer: $r(d + k)$.

Worked example: 7B LLaMA, $r = 8$, all 4 projections, 32 layers

$$32 \times 4 \times 8 \times (4096 + 4096) = 8{,}388{,}608 \approx 8.4\text{M}$$

$$\frac{8.4\text{M}}{7{,}000\text{M}} \approx 0.12\%$$

LoRA vs. Full Fine-Tuning

	LoRA ($r = 8$)	Full fine-tuning
Trainable params	8.4M (0.12%)	7,000M (100%)
A100 training time	$\approx$ 90 min	$\approx$ 40 hr
Inference overhead	Zero (merge BA)	None

RLHF: Three-Stage Alignment Pipeline

THE BIGGER PICTURE

A base model predicts the next token; it does not know what humans *prefer*. RLHF aligns behaviour to human judgement — in finance, the gap between a "plausible" answer and an "accurate, compliant" one is exactly what alignment must close.

Stage 1 — Supervised Fine-Tuning (SFT): Train base model on high-quality human-written demonstrations. Produces initial policy π_{SFT}.

Stage 2 — Reward Model Training: Human annotators rank pairs of responses $(y_w \succ y_l)$ for the same prompt x. Bradley-Terry loss:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}[\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$$

RLHF: Stage 3 — PPO Fine-Tuning

UNDER THE HOOD

Stage 3 — PPO fine-tuning with KL penalty:

$$\mathcal{L}_{\mathrm{RL}}(\theta) = \mathbb{E}_{\pi_\theta}[r_\phi(x, y) - \beta\, \mathrm{KL}(\pi_\theta(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x))]$$

β controls how far the policy can drift from the SFT reference.

Limitations for finance

Reward models can be gamed; PPO is unstable; human annotation is expensive. A single annotator disagreement on "plausible vs. accurate" in a financial context can propagate to thousands of training examples.

Direct Preference Optimisation (DPO)

UNDER THE HOOD

DPO eliminates the reward model entirely. It directly optimises the policy on preference pairs by re-parameterising the reward in terms of log-ratios:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

Direct Preference Optimisation (DPO): Properties

Key properties:

- No separate reward model to train or maintain
- Same preference data as RLHF; simpler single-stage training loop
- Empirically competitive with RLHF on alignment benchmarks
- β still controls implicit KL penalty vs. π_{ref}

Interpretation

DPO increases the log-probability of the preferred response y_w relative to π_{ref} while decreasing the log-probability of y_l — a direct implementation of the preference signal.

FinBERT: Domain-Adapted Encoder for Finance

Architecture: BERT-large continued pre-training on ≈ 4.9B tokens of Reuters news, Financial PhraseBank, and analyst reports (MLM objective).

Classification head:

$$\hat{y} = \mathrm{softmax}(W_c \cdot h_{[\mathrm{CLS}]} + b_c)$$

$h_{[\mathrm{CLS}]}$ is the final-layer representation of the `[CLS]` token.

Empirical results on Financial PhraseBank (all-agree split):

Model	**Accuracy**
Human upper bound	$\approx 97\%$
FinBERT	88.5%
BERT-large (general)	80.7%
Dictionary / rule-based	$\approx 72\%$

The 7.8 pp gain over BERT-large comes almost entirely from domain-specific continued pre-training, not architectural changes.

BloombergGPT: A 50B Finance Decoder

Training corpus: 363B tokens total — 51% Bloomberg financial data (news, filings, transcripts), 49% general web and books.

ALiBi positional encoding:

$$\text{score}(q_i, k_j) = \frac{q_i^\top k_j}{\sqrt{d_k}} - m\,(i - j)$$

where m is a head-specific slope and $(i - j)$ is query-key distance. ALiBi extrapolates more gracefully to sequences longer than those seen in training compared to learned absolute positional embeddings.

Chinchilla lens on BloombergGPT:

	BloombergGPT	Chinchilla optimal
Parameters	50B	50B
Training tokens	363B	$\approx$ 1T ($= 50\text{B} \times 20$)
Status	Under-trained by Chinchilla criterion	

Lesson: even a well-resourced organisation trained a model that violated

Evaluation: Perplexity, Macro-F1, ROUGE

Perplexity (language modelling):

$$\text{PPL} = 2^{H}, \qquad H = -\frac{1}{T}\sum_{t=1}^{T} \log_2 p(x_t \mid x_{<t})$$

Three limitations: not a task metric; gameable by spreading mass; requires token-level probabilities.

Macro-F1 (classification):

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} \frac{2\,P_k R_k}{P_k + R_k}$$

Treats all classes equally regardless of frequency. Appropriate for imbalanced financial datasets (rare event classes matter as much as common ones).

Evaluation: ROUGE and the FinBen Benchmark

ROUGE-N (summarisation):

$$\text{ROUGE-N} = \frac{\sum_{\text{ref}} \sum_{g \in \text{ngrams(ref},N)} \text{count}_{\text{match}}(g)}{\sum_{\text{ref}} \sum_{g \in \text{ngrams(ref},N)} \text{count}(g)}$$

FinBen benchmark

42 datasets, 24 tasks, 21 models. LLMs excel at extraction/classification but struggle on forecasting and decision tasks — exactly what practitioners want most.

Red-Teaming: Financial Failure Modes

Four recurring failure modes identified by red-teaming:

1. **Factual hallucination** — generates plausible but incorrect price forecasts or earnings figures
2. **Temporal hallucination** — cites outdated regulations as current (e.g., pre-2018 MiFID II rules)
3. **Regulatory compliance failure** — provides what appears to be personalised investment advice without required licensing disclosures
4. **Herd-bias amplification** — echoes analyst consensus while suppressing dissenting signals

Empirical illustration: Lopez-Lira & Tang (2023)

A long-short strategy based on ChatGPT sentiment scores achieved a Sharpe ratio > 6 in 2021, declining to ≈ 1 by late 2024 as the signal became widely known and arbitraged away. *Any published alpha source degrades once it is public.*

Four Bias Types in Financial LLMs

Survivorship bias

Training corpora overrepresent successful firms: public companies file EDGAR; bankrupt or never-public firms do not. The model learns a rosier world than the one practitioners inhabit.

Temporal bias

Models trained on historical data are deployed in structurally different market regimes (post-pandemic, rising rates, de-globalisation). Performance observed on historical test sets may not persist.

Four Bias Types in Financial LLMs (continued)

Geographic bias

English-language, US-centric text dominates open web corpora. Non-US financial conventions, currencies, and legal structures are underrepresented.

Analyst consensus bias

Financial commentary echoes dominant sell-side views; contrarian information is rarer in the training distribution and is systematically underweighted.

Hallucination Types and Mitigations

Two hallucination types:

- **Factual hallucination** — fabricated prices, dates, financial figures not present in any training document
- **Regulatory hallucination** — describes rules that are outdated, misquoted, or jurisdiction-mismatched

Hallucination Mitigations

Four principal mitigations:

1. **Retrieval-Augmented Generation (RAG)** — ground outputs in retrieved, verified passages; reduces reliance on parametric memory
2. **Confidence calibration** — train model to express uncertainty; reject or flag responses below a calibrated confidence threshold
3. **Output verification** — secondary model or rule-based checker flags claims inconsistent with a structured knowledge base
4. **Human-in-the-loop** — route low-confidence or high-stakes outputs to expert review before delivery to end users

No single mitigation is sufficient

Production deployments should layer at least two mitigations, with audit trails at every step.

Regulatory Landscape: MiFID II, SR 11-7, EU AI Act

MiFID II Article 37:

- Investment research must be independent from commercial pressures
- LLM contributions require demonstrable *suitability* (analysis matches recipient risk profile) and *explainability*
- Attention weight visualisations are *not sufficient* explanations: they reflect co-occurrence patterns, not causal reasoning chains

SR 11-7 (Federal Reserve Model Risk Management):

- Tiered validation proportional to model risk
- High-risk models (credit, capital, trading) require independent validation, assumption documentation, periodic performance monitoring

EU AI Act — August 2026 deadline for high-risk systems:

- Conformity assessments, technical documentation, human oversight
- Registration in EU database for high-risk financial AI applications
- High-risk: models influencing credit scoring, insurance pricing, investment decisions at scale

Governance: Risk Classification and Four-Component Framework

Three-tier risk classification:

Tier	Examples	Controls required
Low	Internal Q&A, document search	Standard access controls, logging
Medium	Earnings summarisation, first-pass compliance screening	Audit trails, human review sample
High	Credit decisioning, trade recommendation, regulatory filing	Full SR 11-7 validation, EU AI Act registration

Governance: Four-Component Framework

Four-component governance framework:

1. **Risk classification** — assign Low/Medium/High to each use case based on decision autonomy and downstream impact
2. **Audit trails** — capture input prompts, retrieved context, model outputs, and user actions immutably
3. **Monitoring** — track output distributions for drift; alert on anomalous response patterns or accuracy degradation
4. **Incident response** — documented process for rapid mitigation when model failures are detected in production

Lecture 3 Wrap-Up: Key Takeaways

Data: Financial text is 0.5% of the web. EDGAR, transcripts, and Bloomberg are the primary sources. Quality pipelines and MinHash deduplication are essential.

Scaling: Chinchilla says $D^*/N^* \approx 20$. GPT-3 was over-parameterised; BloombergGPT was under-trained. Size alone is not the answer.

Fine-tuning: LoRA achieves 0.12% of 7B parameters for 90 min of training vs. 40 hr full fine-tuning, with zero inference overhead.

Lecture 3 Wrap-Up: Key Takeaways (continued)

Alignment: RLHF (SFT $\rightarrow$ reward model $\rightarrow$ PPO) aligns behaviour. DPO simplifies it to a single supervised objective on preference pairs.

Finance: FinBERT: +7.8 pp over BERT on sentiment. BloombergGPT: under-trained per Chinchilla. FinBen: LLMs still struggle on forecasting.

Governance: MiFID II + SR 11-7 + EU AI Act 2026 require explainability, audit trails, and tiered validation. Attention weights are not explanations.

Lecture 4: AI Agents in Finance

How do we compose trained LLMs into *agents* that plan, use tools, retrieve data, and operate autonomously? Multi-agent orchestration, tool-use APIs, and agent-specific failure modes.

Appendix

Systems & Efficiency: Distributed Training, Kernels, PEFT Variants

Extra / optional material — beyond the core lecture.

Distributed Training: Three Parallelism Types

Type	What is partitioned	Key mechanism
Data parallelism	Batch (each device holds full model)	Ring-AllReduce gradient aggregation
Tensor / model parallelism	Weight matrices column-wise; $W \in \mathbb{R}^{d\times k} \rightarrow [W_1 \vert W_2 \vert \cdots]$	All-gather / all-reduce intra-layer
Pipeline parallelism	Layer groups to successive devices	Micro-batches; bubble fraction $\approx \frac{p-1}{m+p-1}$

ZeRO (Zero Redundancy Optimizer) stages:

- **Stage 1:** partition optimiser states across devices
- **Stage 2:** also partition gradients
- **Stage 3:** also partition model weights $\rightarrow$ near-linear memory reduction with device count

Engineering Efficiency: bfloat16, Checkpointing, FlashAttention

Mixed-precision training (bfloat16):

- Weights stored in bfloat16 (16-bit); fp32 master copy for optimiser
- Halves memory and HBM bandwidth versus fp32
- Loss scaling: multiply loss by s before backward; divide gradients by s to prevent underflow

Gradient checkpointing:

$$\text{Activation memory} = O(L) \xrightarrow{\text{checkpoint every } \sqrt{L}} O(\sqrt{L})$$

Discards intermediate activations; recomputes them during backward pass. Cost: one extra forward pass.

Engineering Efficiency: FlashAttention

FlashAttention: Tiles $QK^\top$ into SRAM-resident blocks; avoids writing the full $N \times N$ attention matrix to HBM. Reduces HBM reads/writes from $O(N^2)$ to $O(N^2/M) \cdot M$ (subquadratic in practice). Exact attention, no approximation.

Combined effect

bfloat16 + checkpointing + FlashAttention enable training models 4–8$\times$ larger on the same hardware compared with naive full-precision training.

Prefix Tuning and Adapters

Prefix tuning:

Learnable prefix vectors are prepended to K and V at every layer:

$$K' = \begin{bmatrix} P_K \\ K \end{bmatrix} \in \mathbb{R}^{(l+n)\times d_k}, \qquad V' = \begin{bmatrix} P_V \\ V \end{bmatrix} \in \mathbb{R}^{(l+n)\times d_v}$$

Only the prefix matrices P_K, P_V (of length l) are trained. Effective for generation tasks; steers the output distribution without touching weights.

Adapters:

A bottleneck module inserted after each sub-layer:

$$h \leftarrow h + W_{\text{up}}\,\text{ReLU}(W_{\text{down}}\,h), \quad W_{\text{down}} \in \mathbb{R}^{d\times m},\ W_{\text{up}} \in \mathbb{R}^{m\times d},\ m \ll d$$

Adds inference latency unless fused.

Why LoRA wins in practice

Zero inference overhead (merge $W_0 + BA$ before serving); multiple adapters swappable for different tasks; QLoRA extends to 4-bit quantised base models.